

Reto Técnico — Senior .NET Engineer

Servicio de Transferencias Idempotente

Tiempo sugerido: **3-4 horas**. Entrega como repositorio Git (o ZIP) con tu historial de commits.

Contexto

Estás construyendo el núcleo de un servicio de pagos. La pieza que te toca es pequeña en superficie pero exigente en los detalles que de verdad importan en producción: **correctitud bajo concurrencia, semántica HTTP, manejo de dinero e idempotencia**.

Buscamos una solución limpia, correcta y bien testeada — no un framework. La sobre-ingenería cuenta en contra.

Objetivo

Implementar una API HTTP en **.NET 10** que procese transferencias de dinero entre cuentas. El estado puede vivir **en memoria** (no se requiere base de datos), pero debe comportarse correctamente bajo carga concurrente.

Usá el **lenguaje y el stack de .NET que prefieras** (C#, F#, etc.) y el framework web que te resulte natural. No prescribimos lenguaje: parte de lo que observamos es qué herramienta elegís y por qué.

Requisitos funcionales

POST /api/transfers

Crea una transferencia entre dos cuentas.

- Header **obligatorio** `Idempotency-Key: <string>`.
- Body: `json { "sourceAccountId": "11111111-1111-1111-1111-111111111111", "destinationAccountId": "22222222-2222-2222-2222-222222222222", "amount": 100.00, "currency": "USD" }`
- Respuesta exitosa: `201 Created` con la transferencia creada y header `Location`.

GET /api/transfers/{id}

Devuelve una transferencia por id, o `404`.

GET /api/accounts/{id}

Devuelve el saldo actual de una cuenta, o `404`. (Para poder verificar resultados.)

Sembrá un par de cuentas al iniciar para poder probar (documentá los ids).

Reglas de negocio y no funcionales

- Dinero:** usá un tipo apropiado para dinero. (Pista: `float / double` no lo es.) Una transferencia es válida solo si ambas cuentas comparten la misma moneda.
- Validaciones:** monto > 0; cuentas existen; misma moneda; origen ≠ destino; fondos suficientes. Cada caso debe mapear al status HTTP correcto.
- Idempotencia:** - Mismo `Idempotency-Key` + mismo body → devolver el **resultado original** sin volver a procesar (el saldo se mueve **una sola vez**). - Mismo `Idempotency-Key` + body distinto → `409 Conflict`. - Falta el header → `400 Bad Request`.
- Concurrencia:** múltiples requests en paralelo **no** deben corromper saldos, permitir sobregiro, ni producirse "lost updates". Si una cuenta con 1000 recibe 50 pedidos concurrentes de 100, exactamente 10 deben tener éxito.

5. **Errores:** respuestas de error como `ProblemDetails` (RFC 9457) con el status adecuado (p. ej. fondos insuficientes → `422` , cuenta inexistente → `404`).

Tests requeridos (parte central de la evaluación)

Como mínimo:

- **Unitarios** del dominio: tipo de dinero (suma/resta, mismatch de moneda) y la regla de retiro/depósito (fondos insuficientes).
- **De integración** sobre la API (`WebApplicationFactory`): happy path, replay idempotente, conflicto de idempotencia, header faltante, fondos insuficientes.
- **Un test de concurrencia** que demuestre que no hay sobregiro ni lost updates.

Componente de IA (obligatorio)

Esperamos que uses un asistente de IA (el que prefieras) durante el reto. **No** evaluamos si lo usaste, sino **cómo**. Incluí un archivo `AI_COLLABORATION.md` que contenga:

1. **Herramienta(s)** que usaste y para qué partes.
2. **2-4 prompts representativos** (los que más valor aportaron), con contexto.
3. **Al menos un caso concreto** en el que la IA propuso algo subóptimo, incorrecto o inseguro y vos lo **detectaste y corregiste**. Explicá cómo lo detectaste.
4. **Una reflexión breve** (5-10 líneas): dónde la IA aceleró el trabajo, dónde estorbó, y qué decidiste hacer vos a mano y por qué.

No hay respuesta "correcta" aquí. Queremos ver criterio: prompting efectivo, revisión crítica del output, validación, y saber cuándo *no* delegar en la IA.

Entregables

- Código fuente compilable (`dotnet build` y `dotnet test` deben pasar).
- `README.md` con cómo correr y probar, decisiones de diseño y trade-offs asumidos.
- `AI_COLLABORATION.md` .
- Historial de commits que muestre cómo fuiste avanzando.

Qué NO buscamos

- Base de datos real, autenticación, microservicios, CQRS, event sourcing, ni capas de abstracción "por las dudas". Resolvé el problema planteado, bien.
- Cobertura de tests del 100%. Queremos tests que prueben lo que importa.

Preguntas

Si algo es ambiguo, tomá una decisión razonable y **documentála** en el README. La capacidad de resolver ambigüedad es parte de lo que evaluamos.